FANTASYREALM

Documentation de l'environnement de développement

SOMMAIRE

1. PRÉSENTATION DE L'ENVIRONNEMENT 2
   1.1. Objectifs 2
   1.2. Outils et technologies utilisés 2
   1.3. Organisation générale 3
2. INSTALLATION ET CONFIGURATION 4
   2.1. Prérequis 4
   2.2. Installation des outils 5
   2.3. Récupération du projet 5
   2.4. Initialisation de la configuration locale 6
   2.5. Construction et démarrage des conteneurs 6
3. ARCHITECTURE DOCKER 7
   3.1. Organisation des services 7
   3.2. Configuration Docker Compose 8
   3.3. Construction de l'image PHP/Apache 9
   3.4. Réseaux Docker 10
3.5. Volumes et persistance des données 10
   3.6. Configuration Apache 11
4. CONFIGURATION DE L'APPLICATION 12
   4.1. Variables d'environnement 12
   4.2. Initialisation de MariaDB 13
4.3. Configuration de MongoDB 14
   4.4. Configuration de l'envoi d'e-mails 14
5. UTILISATION ET MAINTENANCE 15
   5.1. Commandes courantes 15
   5.2. Vérification du fonctionnement 17
   5.3. Gestion des modifications 17
   5.4. Résolution des problèmes courants 18
   5.5. Gestion des versions 19
6. CONCLUSION 19

1. PRÉSENTATION DE L'ENVIRONNEMENT

1.1. Objectifs

Le projet FantasyRealm est une application web dynamique permettant aux utilisateurs de créer, personnaliser, gérer et partager des personnages fantastiques.

Son fonctionnement nécessite plusieurs composants techniques : un serveur web, un environnement PHP, une base de données relationnelle et une base de données non relationnelle.

Afin de simplifier leur installation et de garantir la cohérence de leur fonctionnement, l'environnement de développement repose sur Docker Desktop et Docker Compose.

Cette solution permet d'exécuter les différents services dans des conteneurs isolés tout en conservant leurs capacités de communication.

L'utilisation de Docker présente plusieurs avantages :

- Centraliser la configuration des services nécessaires à l'application.
- Limiter les différences entre les environnements.
- Éviter l'installation individuelle des serveurs et bases de données sur le système hôte.
- Simplifier le démarrage et l'arrêt des composants.
- Faciliter la mise à jour des images et des dépendances.
- Préserver les données grâce à des volumes persistants.
- Préparer le déploiement de l'application sur un serveur.

L'objectif est de disposer d'un environnement reproductible, adapté au développement et à la maintenance de FantasyRealm.

1.2. Outils et technologies utilisés

Outil ou technologie	Utilisation
Windows	Système d'exploitation du poste de travail
Docker Desktop	Gestion des conteneurs
Docker Compose	Configuration et orchestration des services

Visual Studio Code	Développement et modification du code
Git	Gestion des versions
GitHub	Hébergement du dépôt de code
Git Bash	Exécution des commandes et scripts Bash
Apache	Serveur HTTP
PHP 8.5	Exécution des traitements backend
MariaDB 11.4	Stockage des données relationnelles
MongoDB 7.0	Stockage des journaux et historiques
Composer 2	Gestion des dépendances PHP
Navigateur web	Exécution et test des interfaces

L'application utilise HTML5, CSS3 et JavaScript natif pour le frontend.

Les traitements backend sont réalisés en PHP. Les échanges entre le navigateur et les API sont effectués à l'aide de requêtes HTTP et de données JSON.

L'application ne nécessite pas l'installation d'un serveur WAMP ou XAMPP.

Le serveur Apache, PHP, MariaDB et MongoDB sont exécutés directement dans des conteneurs Docker.

1.3. Organisation générale

L'environnement repose sur trois services principaux.

Le service web

Il contient Apache, PHP et les dépendances nécessaires à l'exécution de FantasyRealm.

Il fournit les pages HTML, les fichiers CSS et JavaScript, ainsi que les API PHP.

Le service db

Il utilise MariaDB pour conserver les utilisateurs, les rôles, les personnages, les apparences, les équipements, les avis et les demandes de contact.

Le service mongodb

Il assure le stockage documentaire des journaux d'activité et des historiques de personnalisation.

Ces services sont définis dans les fichiers Docker Compose du projet.

En développement local, le navigateur accède au serveur web par l'adresse localhost sur le port 8080.

Les bases de données restent accessibles uniquement aux services autorisés sur le réseau Docker privé.

2. INSTALLATION ET CONFIGURATION

2.1. Prérequis

L'installation locale nécessite :

- Un ordinateur compatible avec Docker Desktop.
- Un système Windows prenant en charge les conteneurs Linux.
- Docker Desktop installé et démarré.
- Git et Git Bash.
- Visual Studio Code.
- Une connexion Internet pour récupérer le dépôt et les images Docker.
- Un navigateur web récent.
- Un port 8080 disponible sur la machine locale.

Sous Windows, Docker Desktop peut utiliser le sous-système Windows pour Linux, appelé WSL 2.

Celui-ci doit être configuré lorsque le moteur Docker choisi en dépend.

Un espace disque suffisant est également nécessaire pour les images Docker, les dépendances et les volumes de données.

2.2. Installation des outils

Docker Desktop

Docker Desktop permet de télécharger, construire, exécuter et superviser les conteneurs.

Il doit être installé depuis le site officiel : https://www.docker.com/products/docker-desktop/

Après installation, le moteur Docker doit être démarré.

La disponibilité des commandes peut être vérifiée depuis Git Bash :

docker --version

docker compose version

Ces commandes permettent de contrôler que Docker et Docker Compose sont accessibles.

Visual Studio Code

Visual Studio Code est utilisé pour consulter et modifier les différents fichiers du projet.

Il permet notamment de travailler sur les fichiers HTML, CSS, JavaScript, PHP, SQL, YAML et Dockerfile.

L'éditeur peut être téléchargé depuis : https://code.visualstudio.com/

Git

Git permet de suivre les modifications apportées aux fichiers et de conserver l'historique du développement.

Git Bash fournit également un terminal permettant d'utiliser les scripts Bash présents dans FantasyRealm.

Site officiel : https://git-scm.com/

L'installation peut être vérifiée avec : *git --version*

2.3. Récupération du projet

Le code source est hébergé dans le dépôt GitHub suivant :
https://github.com/IbbiBoy2/FantasyRealm/

La branche de référence pour le développement est dev.

Pour récupérer le projet depuis Git Bash, exécuter :

git clone -b dev https://github.com/IbbiBoy2/FantasyRealm.git

Se placer ensuite dans le répertoire du projet :

cd FantasyRealm

Le dossier peut être ouvert dans Visual Studio Code : *code .*

Le dépôt comporte notamment les répertoires Frontend, Backend, docker et scripts.

Les fichiers Docker Compose se trouvent à la racine du projet.

2.4. Initialisation de la configuration locale

Avant le premier démarrage des services, FantasyRealm nécessite la création de fichiers de configuration contenant les informations de connexion aux bases de données.

Cette opération est automatisée grâce au script : ***scripts/init-local.sh***

Depuis la racine du projet, exécuter : *bash scripts/init-local.sh*

Le script crée deux fichiers dans le répertoire .local :

.local/compose.env

Ce fichier contient les mots de passe utilisés par les services MariaDB et MongoDB, ainsi que le chemin du fichier de configuration backend.

.local/backend.env

Ce fichier contient les informations de connexion utilisées par PHP pour accéder aux bases de données.

Les mots de passe sont générés automatiquement à l'aide d'OpenSSL.

Le script utilise également umask 077 afin de limiter les permissions des fichiers créés.

Si les fichiers existent déjà, le script ne les remplace pas.

Cette précaution évite d'écraser accidentellement les paramètres d'un environnement existant.

Les dossiers .local et .private sont exclus du versionnement Git.

Les fichiers contenant des secrets ne doivent jamais être ajoutés au dépôt.

2.5. Construction et démarrage des conteneurs

Une fois la configuration locale préparée, le démarrage de l'application s'effectue avec :

bash scripts/start-local.sh

Ce script exécute Docker Compose en combinant deux fichiers :

- ***compose.yml*** : configuration commune des services.
- ***compose.local.yml*** : paramètres spécifiques au développement local.

La commande Docker réellement exécutée est :

docker compose \

--env-file .local/compose.env \

-f compose.yml \

-f compose.local.yml \

up -d --build --wait

L'option --build demande la construction de l'image applicative.

L'option -d permet d'exécuter les conteneurs en arrière-plan.

L'option --wait demande d'attendre que les services atteignent leur état de démarrage ou de santé attendu.

Lors de la première exécution, Docker télécharge les images nécessaires, construit l'image PHP et installe les dépendances du projet.

Une fois le démarrage terminé, l'application est accessible à l'adresse :

http://localhost:8080/FantasyRealm/Frontend/Html/index.html

Le navigateur doit accéder à l'application au travers d'Apache, et non en ouvrant directement les fichiers HTML depuis l'explorateur Windows.

En effet, les pages utilisent des fonctionnalités JavaScript qui communiquent avec les API PHP.

3. ARCHITECTURE DOCKER

3.1. Organisation des services

Docker Compose permet de définir plusieurs services dans des fichiers YAML.

Le fichier ***compose.yml*** constitue la configuration commune.

Il définit trois services : web, db et mongodb.

Cette séparation permet d'isoler les différentes responsabilités de l'application.

Le service web communique avec MariaDB et MongoDB au moyen des noms de services Docker.

Les adresses db et mongodb sont donc utilisées dans la configuration backend plutôt que localhost.

Chaque service dispose de son propre environnement d'exécution et de ses paramètres.

3.2. Configuration Docker Compose

Fichier compose.yml

Le fichier principal définit les images, les volumes, les dépendances et les réseaux communs.

Le service web est construit à partir d'un Dockerfile personnalisé.

Il utilise un fichier de configuration backend privé et partage un volume dédié aux images des accessoires.

Son démarrage dépend de la disponibilité du service MariaDB et du démarrage de MongoDB.

MariaDB dispose d'un contrôle de santé permettant à Docker de vérifier son initialisation avant le démarrage de l'application.

MongoDB est configuré avec une authentification et un volume persistant.

Les trois services utilisent une politique de redémarrage unless-stopped.

Fichier compose.local.yml

Ce fichier ajoute les paramètres spécifiques au développement.

Il expose le serveur web uniquement sur l'interface locale :

ports:

- *"127.0.0.1:8080:80"*

Le port 8080 est utilisé par le navigateur sur la machine hôte.

Le port 80 correspond au port HTTP d'Apache à l'intérieur du conteneur.

Cette configuration évite de publier volontairement le service sur l'ensemble des interfaces réseau de l'ordinateur.

Le fichier ajoute également des montages du code source.

Le répertoire Frontend est monté en lecture-écriture.

Les répertoires Backend/api et Backend/config sont montés en lecture seule depuis le conteneur.

Ces montages permettent au serveur de consulter les fichiers présents sur le poste de développement, sans devoir reconstruire l'image après chaque modification ordinaire du frontend ou de ces fichiers PHP.

Fichier compose.prod.yml

Un troisième fichier prépare les différences de configuration nécessaires à la production.

Il prévoit notamment le rattachement du service web au réseau externe proxy.

Un alias réseau fantasyrealm-ecf-web est défini pour permettre à un reverse proxy de communiquer avec l'application.

Ce fichier est destiné à compléter ***compose.yml*** sur le serveur.

La procédure détaillée de mise en production fera l'objet d'une documentation spécifique.

3.3. Construction de l'image PHP/Apache

Le fichier docker/Dockerfile définit l'image applicative.

Il utilise comme base :

FROM php:8.5.12-apache-trixie

Cette image fournit Apache et PHP.

Des dépendances système et extensions PHP supplémentaires sont installées.

L'extension pdo_mysql permet la communication avec MariaDB.

L'extension MongoDB est installée via PECL, puis activée.

Le gestionnaire de dépendances Composer est récupéré depuis l'image officielle composer:2.

Les fichiers ***composer.json*** et ***composer.lock*** sont ensuite copiés dans l'image afin d'installer les dépendances PHP.

L'installation utilise notamment l'option --no-dev, qui exclut les dépendances réservées au développement.

Les répertoires frontend, API et configuration sont copiés dans l'image.

La configuration Apache personnalisée est également intégrée.

Les modules Apache headers et rewrite sont activés.

Enfin, les droits nécessaires sont attribués à l'utilisateur du serveur web et le port HTTP 80 est exposé dans le conteneur.

Cette organisation permet de construire une image applicative reproductible à partir du code et des dépendances versionnées.

3.4. Réseaux Docker

Deux réseaux sont utilisés pour le développement local.

Réseau private

Le réseau private permet la communication entre les trois services de l'application.

Il est défini avec l'option :

networks:

private:

internal: true

Cette configuration isole le réseau des communications extérieures directes.

Les bases de données ne disposent d'aucune publication de port vers l'ordinateur hôte.

Réseau frontend

Le réseau frontend est utilisé par le service web en environnement local.

Il utilise le pilote bridge.

Le conteneur web dispose donc d'une connexion au réseau privé des bases de données et d'une connexion au réseau frontend.

Le navigateur accède uniquement au port HTTP publié sur l'interface locale de la machine.

Cette organisation limite les surfaces d'exposition des composants internes.

3.5. Volumes et persistance des données

Docker Compose définit trois volumes nommés.

Volume	Utilisation
mariadb_data	Conservation des données MariaDB
mongodb_data	Conservation des données MongoDB
accessory_images	Conservation des images des accessoires téléversés

Ces volumes permettent de conserver les données lorsque les conteneurs sont arrêtés ou recréés.

La suppression d'un conteneur ne provoque donc pas automatiquement la suppression des données enregistrées.

Les volumes sont distincts du système de fichiers temporaire des conteneurs.

Cette séparation facilite les mises à jour de l'environnement et la maintenance des services.

Il faut toutefois distinguer l'arrêt des conteneurs de la suppression volontaire des volumes.

La commande docker compose down -v supprime également les volumes associés à la composition et peut entraîner la perte des données locales.

Elle ne doit donc pas être utilisée pour un simple arrêt de l'environnement.

3.6. Configuration Apache

La configuration du serveur HTTP se trouve dans : ***docker/apache.conf***

Elle définit la racine documentaire et les répertoires accessibles depuis le navigateur.

Le fichier prévoit une redirection de la racine de l'application vers la page d'accueil de FantasyRealm.

L'accès public est autorisé aux ressources du répertoire Frontend ainsi qu'aux points d'entrée du répertoire Backend/api.

Le reste de l'arborescence n'est pas rendu publiquement accessible par défaut.

Cette configuration permet notamment d'empêcher l'accès HTTP direct aux fichiers internes de configuration backend.

Les options de listing des répertoires sont désactivées.

Plusieurs en-têtes HTTP de sécurité sont également configurés, notamment :

- X-Content-Type-Options: nosniff
- Referrer-Policy: strict-origin-when-cross-origin

Apache utilise des fichiers spécifiques pour les journaux d'accès et d'erreurs de FantasyRealm.

Les paramètres de sécurité complémentaires de l'application seront détaillés dans la documentation technique.

4. CONFIGURATION DE L'APPLICATION

4.1. Variables d'environnement

L'application utilise des variables d'environnement afin de séparer le code source des informations de configuration.

Un exemple est disponible dans : ***Backend/.env.example***

Le fichier réel utilisé par l'environnement Docker local est généré dans : ***.local/backend.env***

Ce dernier est monté en lecture seule dans le conteneur web à l'emplacement attendu par l'application PHP.

Les principales variables concernent :

Variable	Fonction
DB_HOST	Nom du service MariaDB
DB_PORT	Port MariaDB
DB_NAME	Nom de la base
DB_USER	Compte applicatif MariaDB
DB_PASSWORD	Mot de passe MariaDB
MONGODB_URI	Chaîne de connexion MongoDB
MONGODB_DATABASE	Nom de la base MongoDB

MAIL_HOST	Serveur SMTP
MAIL_PORT	Port SMTP
MAIL_USERNAME	Identifiant SMTP
MAIL_PASSWORD	Mot de passe SMTP
MAIL_FROM	Adresse d'expédition
MAIL_FROM_NAME	Nom de l'expéditeur
MAIL_CONTACT_TO	Adresse de réception des contacts
APP_PUBLIC_URL	Adresse utilisée pour construire les liens publics

Les fichiers contenant les valeurs réelles ne doivent pas être publiés.

Les fichiers ***.gitignore*** et ***.dockerignore*** contribuent à éviter l'intégration accidentelle de certains secrets et fichiers locaux au dépôt ou à l'image Docker.

4.2. Initialisation de MariaDB

La base MariaDB utilise l'image officielle : *image: mariadb:11.4*

Les identifiants nécessaires sont transmis par des variables d'environnement.

L'image initialise la base fantasyrealm ainsi que le compte applicatif prévu.

Deux fichiers SQL sont montés dans le répertoire d'initialisation de MariaDB :

- ***Backend/Database/shema.sql***
- ***Backend/Database/seed.sql***

Le premier définit la structure des tables et leurs relations.

Le second insère les données initiales, notamment les rôles USER, EMPLOYEE et ADMIN.

Ces scripts sont exécutés automatiquement lors de l'initialisation d'un répertoire de données MariaDB vide.

Ils ne sont pas rejoués automatiquement à chaque démarrage des conteneurs.

Cette distinction est importante lors des modifications du schéma de base de données.

La mise à jour d'un environnement existant devra être réalisée au moyen de scripts adaptés ou d'une procédure de migration contrôlée, sans supprimer les données.

4.3. Configuration de MongoDB

MongoDB utilise l'image officielle : *image: mongo:7.0*

Son compte administrateur est initialisé à l'aide des variables d'environnement Docker.

Le backend utilise une chaîne de connexion contenant le nom du service mongodb.

MongoDB est utilisé pour conserver certaines données non relationnelles de FantasyRealm.

Les collections utilisées comprennent notamment :

activity_logs

Cette collection conserve des événements liés aux actions réalisées dans l'application, particulièrement les opérations de gestion et de modération.

character_history

Cette collection permet de conserver l'historique de certaines opérations sur les personnages et des instantanés de leur personnalisation.

Le stockage MongoDB est persistant grâce au volume mongodb_data.

Les documents enregistrés restent ainsi conservés lorsque le conteneur est redémarré.

4.4. Configuration de l'envoi d'e-mails

FantasyRealm utilise PHPMailer pour l'envoi des notifications électroniques.

La configuration SMTP est définie dans les variables d'environnement du backend.

L'application peut utiliser le serveur SMTP Gmail :

MAIL_HOST=smtp.gmail.com

MAIL_PORT=465

MAIL_USERNAME=adresse@example.com

MAIL_PASSWORD=MOT_DE_PASSE_APPLICATION

MAIL_FROM=adresse@example.com

MAIL_FROM_NAME=FantasyRealm

MAIL_CONTACT_TO=adresse@example.com

Les valeurs présentées ci-dessus sont fictives.

Le mot de passe doit correspondre au mécanisme d'authentification autorisé par le fournisseur SMTP.

Les informations réelles sont conservées uniquement dans le fichier d'environnement privé.

La configuration SMTP est utilisée pour les fonctionnalités telles que la récupération de mot de passe, les notifications de modération et les demandes de contact.

Un message transmis depuis le formulaire de contact est également conservé dans MariaDB afin de rester consultable depuis le tableau de bord des employés.

La configuration des e-mails n'est pas générée automatiquement par le script d'initialisation locale et doit être ajoutée dans le fichier privé.

Après modification de ce fichier, le conteneur web devra être recréé si nécessaire afin de charger la nouvelle configuration.

5. UTILISATION ET MAINTENANCE

5.1. Commandes courantes

Les scripts du projet permettent de simplifier les opérations quotidiennes.

Initialiser la configuration

bash scripts/init-local.sh

Démarrer l'environnement

bash scripts/start-local.sh

Arrêter les conteneurs

bash scripts/stop-local.sh

L'arrêt normal conserve les volumes et les données enregistrées.

Vérifier les services

```
docker compose \
  --env-file .local/compose.env \
  -f compose.yml \
```

*-f compose.local.yml *

ps

Consulter les journaux

*docker compose *

*--env-file .local/compose.env *

*-f compose.yml *

*-f compose.local.yml *

logs -f

Cette commande permet de consulter les événements produits par les différents services.

Pour cibler uniquement le serveur web, ajouter web à la fin de la commande.

Accéder au conteneur web

*docker compose *

*--env-file .local/compose.env *

*-f compose.yml *

*-f compose.local.yml *

exec web bash

Cette commande ouvre un terminal dans le conteneur Apache/PHP.

Accéder à MariaDB

*docker compose *

*--env-file .local/compose.env *

*-f compose.yml *

*-f compose.local.yml *

exec db mariadb -u root -p

Le mot de passe root est défini dans ***.local/compose.env.***

Les données sensibles ne doivent pas être copiées dans la documentation.

5.2. Vérification du fonctionnement

Après le démarrage de l'environnement, Docker Desktop permet de visualiser les conteneurs associés à FantasyRealm.

Le conteneur web, MariaDB et MongoDB doivent être actifs.

Le contrôle de santé MariaDB doit atteindre l'état healthy.

L'application peut ensuite être ouverte depuis un navigateur.

Les premières vérifications peuvent concerner :

- L'affichage de la page d'accueil.
- Le chargement des styles et des images.
- La navigation entre les pages.
- L'inscription et la connexion.
- La consultation de la galerie.
- La communication avec les API PHP.
- L'enregistrement des données MariaDB.
- Les opérations utilisant MongoDB.
- Les notifications SMTP lorsque le service est configuré.

Le bon démarrage des conteneurs ne suffit pas à confirmer le fonctionnement de toutes les fonctionnalités métier.

Celles-ci doivent être vérifiées indépendamment.

5.3. Gestion des modifications

L'environnement local utilise des montages de répertoires permettant de faciliter le développement.

Les modifications ordinaires des fichiers frontend et des fichiers PHP montés depuis le poste sont disponibles dans le conteneur sans reconstruction systématique.

En revanche, les modifications apportées au Dockerfile, aux dépendances Composer ou aux éléments copiés uniquement lors de la construction nécessitent une nouvelle construction de l'image.

Le script ***start-local.sh*** permet de relancer le processus avec l'option --build.

En cas de modification des variables d'environnement utilisées par le processus du conteneur, une recréation du service peut être nécessaire.

La configuration doit toujours être vérifiée après une modification importante.

5.4. Résolution des problèmes courants

Docker Desktop n'est pas démarré

Si les commandes Docker ne parviennent pas à communiquer avec le moteur, vérifier que Docker Desktop est ouvert et que son moteur est actif.

Le port 8080 est déjà utilisé

Le port local 8080 doit être disponible pour permettre l'accès à Apache.

Il convient d'identifier l'application occupant ce port ou d'adapter le mapping dans la configuration locale.

MariaDB ne démarre pas

Consulter les journaux du service db et vérifier sa configuration.

Un problème d'initialisation peut provenir d'un volume existant, d'identifiants incompatibles ou d'une erreur dans les scripts SQL.

L'application ne parvient pas à accéder à la base

Vérifier les informations de connexion dans le fichier d'environnement backend ainsi que l'état des services.

Le nom du service MariaDB utilisé par PHP est db, et non localhost.

Une modification du code n'apparaît pas

Vérifier que le fichier concerné est bien monté dans le conteneur ou inclus dans l'image applicative.

Selon le type de modification, un rechargement du navigateur, une recréation du conteneur ou une reconstruction de l'image pourra être nécessaire.

Un e-mail n'est pas reçu

Vérifier les paramètres SMTP, la disponibilité du serveur de messagerie et les journaux backend.

La réussite de la connexion SMTP ne garantit pas nécessairement la remise du message dans la boîte de réception du destinataire.

Les données ont disparu

Vérifier que les volumes Docker n'ont pas été supprimés.

La suppression des volumes avec down -v peut provoquer la disparition des bases locales.

Les sauvegardes restent nécessaires pour protéger les données importantes.

5.5. Gestion des versions

Le projet utilise Git et GitHub pour conserver l'historique des modifications.

Le développement courant est effectué sur la branche dev.

Les commandes principales permettent de vérifier l'état du dépôt : *git status*

Afficher les modifications : *git diff*

Consulter l'historique : *git log --oneline*

Créer un commit :

git add .

git commit -m "Description de la modification"

Publier les modifications validées : *git push origin dev*

Avant chaque commit, les fichiers ajoutés doivent être contrôlés afin d'éviter de publier des données personnelles, des mots de passe ou des fichiers d'environnement.

La gestion des tâches et du suivi de projet est organisée avec Trello : https://trello.com/b/OaO0sxyk

L'organisation des tâches et l'utilisation de Git seront développées dans la documentation de gestion de projet.

6. CONCLUSION

L'environnement de développement FantasyRealm repose sur Docker Desktop et Docker Compose.

Cette architecture permet de regrouper les différents composants nécessaires au fonctionnement de l'application tout en séparant leurs responsabilités.

Le serveur Apache/PHP assure l'exécution des interfaces et des traitements métier.

MariaDB permet le stockage des données relationnelles, tandis que MongoDB conserve les informations de journalisation et d'historisation.

Les fichiers Docker Compose, les scripts d'initialisation et les variables d'environnement permettent de reproduire l'installation sur un poste disposant des outils nécessaires.

Les réseaux privés, les volumes persistants et la séparation des fichiers sensibles contribuent à la fiabilité et à la sécurisation de l'environnement.

Cette organisation facilite les opérations de développement, de maintenance et de préparation du déploiement de FantasyRealm.